

Redux

RUJUAN XING



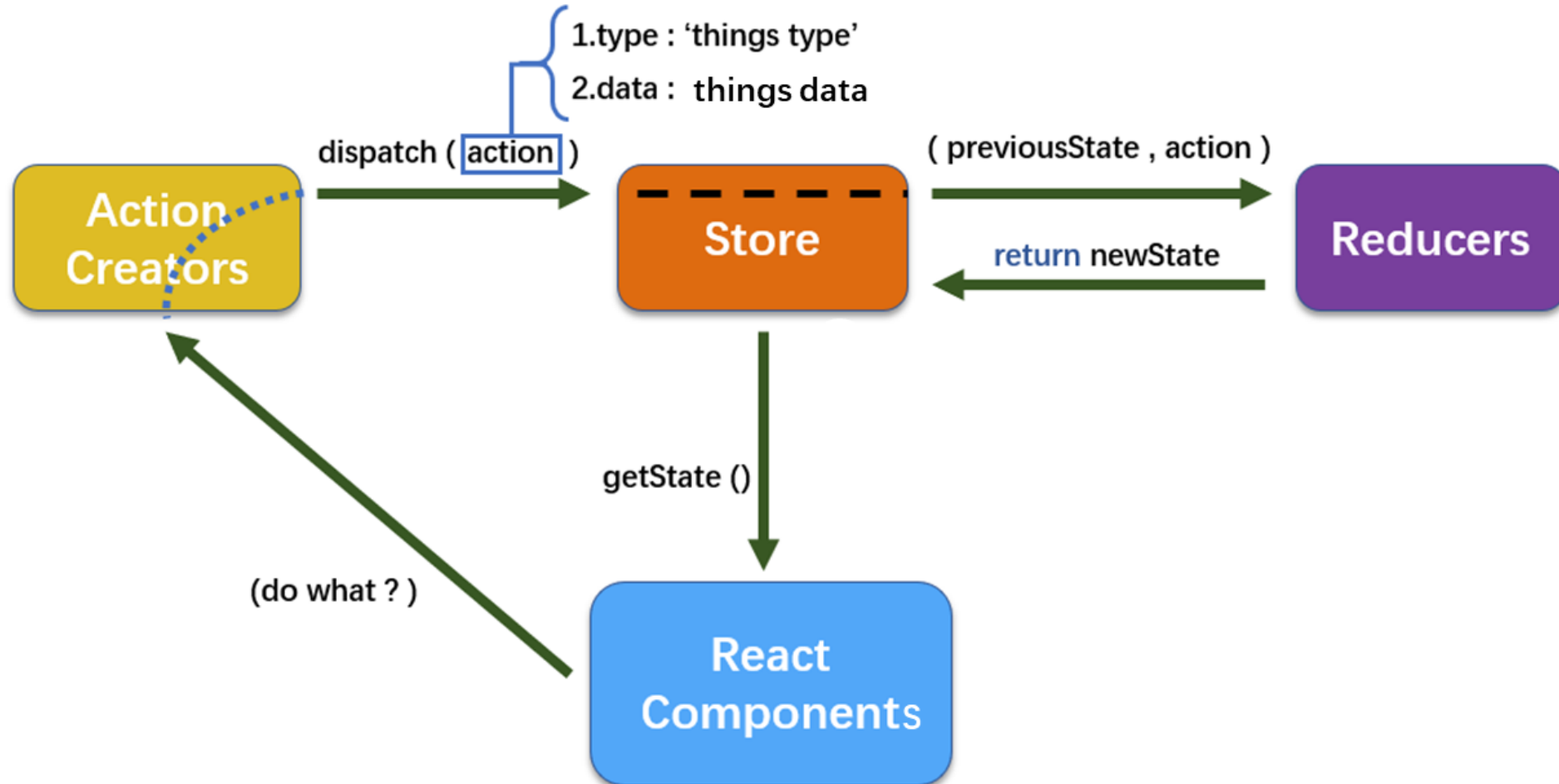
What is Redux?

- Redux is an open-source JavaScript library used for **managing the state** of a web application.
- It can be used with other JavaScript libraries and frameworks like Angular, React, Vue, etc
- Manage the state shared across multiple components within React applications in a centralized manner

When should we use Redux?

1. At any given time, the state of a specific component must be available and shared with other components.
2. One component needs to change the state of another component (communication)
3. General guideline: If you don't need to use, don't use it. If encounter lots of challenges without using it, use it.

Redux workflow



Redux Core Components

Actions: Plain JavaScript objects that describe a change in the application's state. Actions are dispatched to the Redux store and trigger the state update.

- The object has two properties: type and data.

ActionCreators: Used to create Actions.

Reducers: Functions that specify how the application's state changes in response to actions. Reducers calculate the new state based on the previous state and the action being dispatched.

- It's used to init state and update the state
- It's a pure function which produces new state based on previous state and action.

Store: The central repository for the state of your application. It holds the entire state tree of your application. It connects ActionCreators, Reducers and React Components.

Redux Core APIs

1. Redux: createStore() – create store based on reducers. (will replace with new one later)
2. store instance – the center of the redux
 - 1) store.getState() – get maintained state in redux
 - 2) store.dispatch(action) - send an action to the Redux store
 - 3) store.subscribe(render) - provided by the Redux store that allows you to register a callback function to be invoked whenever the state of the store changes.

Async Action

There are two types of action: Synchronous action and Asynchronous Action.

- Synchronous action: Redux can only handle Synchronous actions by default.
 - Plain object: {type: "", data: ""}
- Asynchronous action:
 - Function – only function can perform async tasks(setTimeout, fetch, etc), other types like array, string, they cannot perform.
 - Example: () => {setTimeout()}

Redux thunk

redux-thunk is a middleware for Redux for asynchronous action.

Handling Asynchronous Actions: Redux is primarily designed for handling synchronous actions, where an action triggers a state change immediately. However, in real-world applications, you often need to deal with asynchronous actions like making network requests (e.g., fetching data from an API). This is where redux-thunk comes in.

```
npm install redux-thunk
```

```
applyMiddleware() – apply other middlewares in redux
```


React Redux

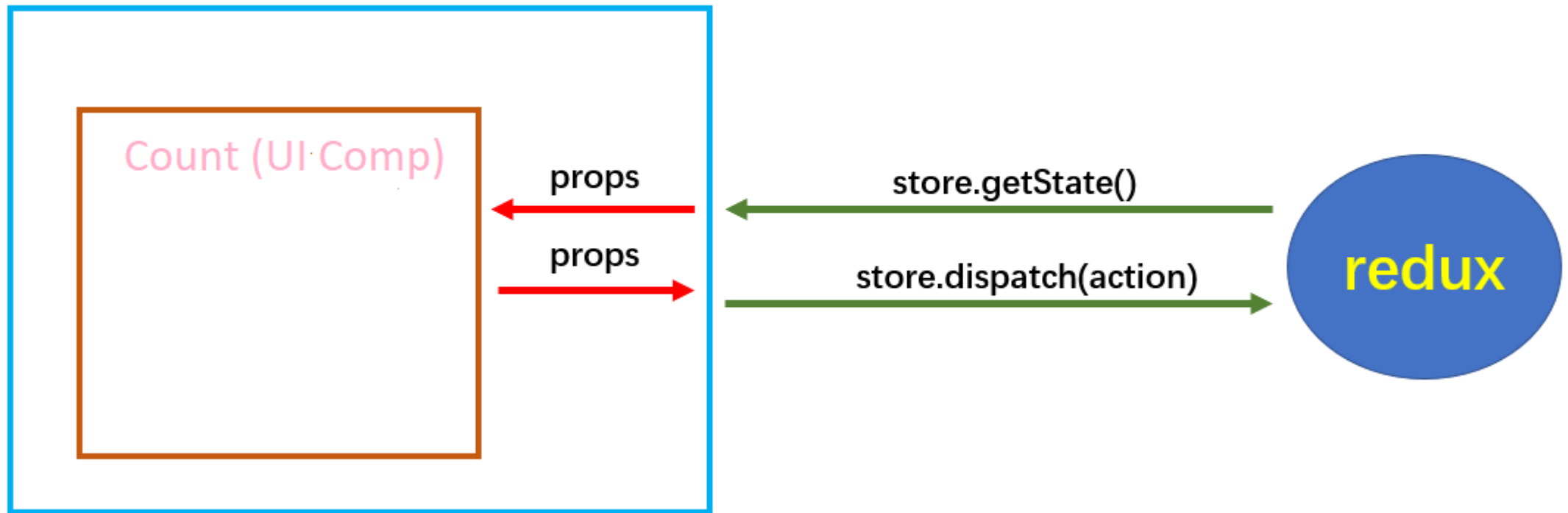
React Redux is a library that integrates the Redux state management framework with React.

React Redux categories components:

1. UI component:
 - 1) Only responsible for UI
 - 2) Receive data via props
 - 3) No redux APIs used
 - 4) Store in `components` folder generally
2. Container Component:
 - 1) Responsible for maintaining state, not UI
 - 2) Contains Redux Logic
 - 3) Store in `containers` folder generally

React Redux

Count (Container Comp)



React Redux

1. All UI components should be wrapped in a container component, parent – child relationship
2. Container component is responsible for interact with Redux
3. Container component transfer two things into UI component only via props
 - 1) State store in redux
 - 2) The functions used to operate state store in redux

`npm install react-redux`

React Redux APIs

1. Provider: provide store for all component

```
<Provider store={store}>  
  <App />  
</Provider>
```

2. connect: wrap UI component to get a container component

```
import { connect } from 'react-redux'  
connect(  
  mapStateToProps,  
  mapDispatchToProps  
) (Counter)
```

- 1) mapStateToProps: map state stored in redux to props in UI component
- 2) mapDispatchToProps: map the function which dispatch action into props into UI component

Pure function

same input, same output

1. Cannot change the value of arguments
2. Cannot have side effect, like network request(fetch), etc
3. Cannot call new Data(), Math.random, etc

Reducer function in Redux must be a pure function

Modernizing Redux Logic with Redux Toolkit

1. Install & Uninstall

- 1) `npm install @reduxjs/toolkit`
- 2) `npm uninstall redux redux-thunk`

2. Reducers and Actions with `createSlice`

- 1) Redux Toolkit's `createSlice` API was designed to eliminate all the "boilerplate" with writing reducers, actions, and immutable updates!

```
import {createSlice} from "@reduxjs/toolkit";

const initialState = 0;

const countSlice = createSlice({
  name: 'count',
  initialState,
  reducers: {
    increment(state, action){
      console.log(action);
      const {payload} = action;
      return state + payload;
    },
    decrement(state, action){
      const {payload} = action;
      return state - payload;
    }
  }
});

export const {increment, decrement} = countSlice.actions;
export default countSlice.reducer;
```

Redux Toolkit - Async

```
const delayOperation = (value, delay) => {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve(value);  
    }, delay);  
  });  
};  
  
export const addAsync = createAsyncThunk(  
  'count/addAsync',  
  async ({ value, delay }) => {  
    const response = await delayOperation(value, delay);  
    return response;  
  }  
);
```

```
const countSlice = createSlice({  
  
  extraReducers: (builder) => {  
    builder  
      .addCase(addAsync.fulfilled, (state, action) => {  
        state += action.payload;  
        return state;  
      });  
  },  
});
```

```
const addAsyncHandler = () => {  
  store.dispatch(addAsync({ value: selectRef.current.value * 1, delay: 2000 }));  
}
```

React Redux Toolkit

Redux Toolkit is an opinionated, official package from the Redux team that simplifies and streamlines the usage of Redux in your React applications. It's designed to make it easier to write Redux code by providing a set of tools and abstractions to reduce boilerplate code and common patterns associated with Redux.

It has built in `redux-thunk` and `redux-devtools-extension`

The choice between React Redux Toolkit and React Redux depends on your specific use case and your familiarity with Redux. If you're new to Redux or prefer a more opinionated and streamlined approach, React Redux Toolkit is a great choice. It simplifies many aspects of Redux, reduces boilerplate, and helps you follow best practices.

If you have experience with Redux and need maximum flexibility and control over your Redux setup, or if you're working on a project with non-standard requirements, React Redux might be the better option for you.

In practice, both can coexist, and you can even start with React.

<https://redux.js.org/tutorials/fundamentals/part-8-modern-redux>

React Hook: useReducer

The `useReducer` Hook is similar to the `useState` Hook.

It allows for custom state logic.

If you find yourself keeping track of multiple pieces of state that rely on complex logic, `useReducer` may be useful.

```
const [state, dispatch] = useReducer(<reducer>, <initialState>)
```

The `reducer` function contains your custom state logic and the `initialState` can be a simple value but generally will contain an object. The `useReducer` Hook returns the current `state` and a `dispatch` method.

useReducer vs Redux

Prefer useReducer if we have:

- Small to medium-sized applications with relatively simple state management needs
- Applications that only need to manage local state within a component or a few components

Prefer Redux Toolkit if we have:

- Large-scale applications with complex state management needs. Redux is more flexible and scalable than useReducer, and it has a larger ecosystem of tools and plugins.
- Applications that need to manage global state across multiple components and/or pages
- Applications that require middleware and advanced tools for debugging and performance optimization

Main Point

Frameworks make Web development easier and more effective by providing a secure and reliable foundation on which to build upon.

The simplest form of awareness, Transcendental Consciousness, provides a strong foundation for a rewarding and successful life.